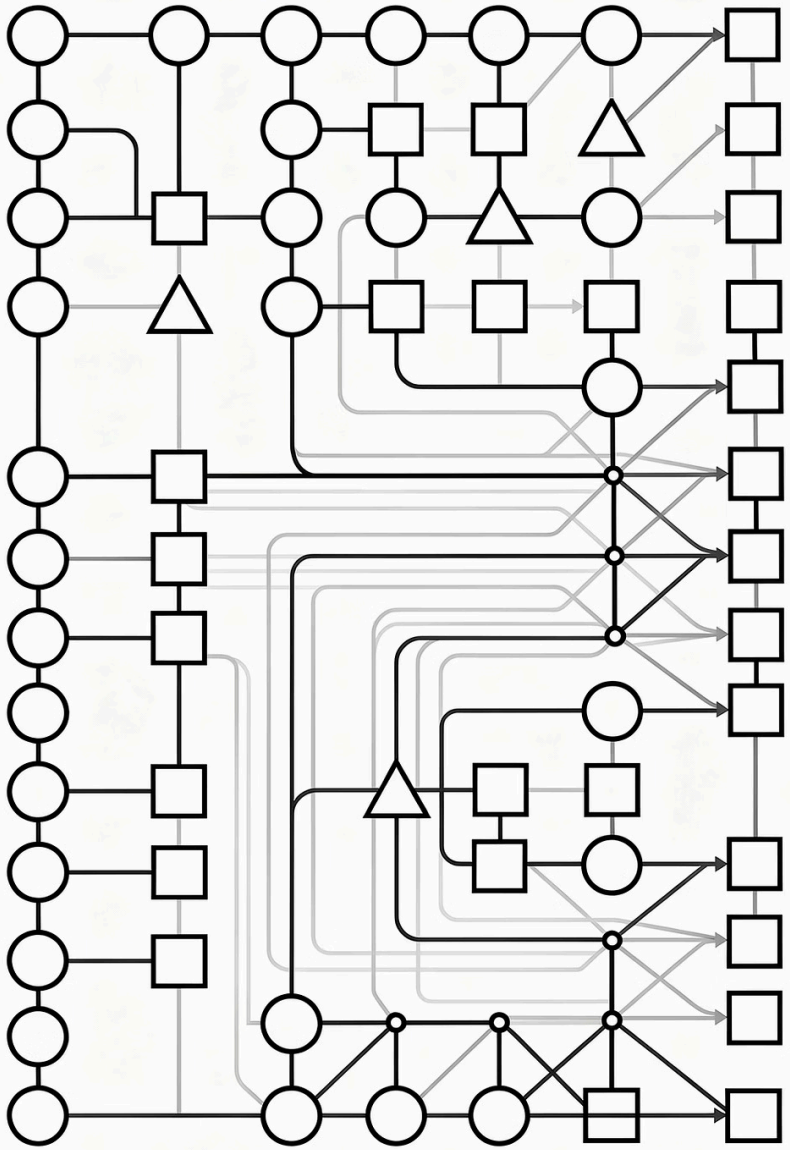


El Motor de Inferencia

La unidad lógica que da vida a los sistemas expertos — cómo el conocimiento se convierte en razonamiento automatizado.

SISTEMAS EXPERTOS · HORA 2

¿Qué es el Motor de Inferencia?

Definición

Es la **unidad lógica central** que dirige la ejecución del sistema experto. Su función principal es aplicar las reglas almacenadas en la base de conocimientos sobre los hechos conocidos para **deducir nueva información** o tomar decisiones de manera automática.

Sin el motor de inferencia, las reglas serían simplemente datos estáticos sin capacidad de razonamiento.

La Separación Vital

En un Sistema Experto, el **conocimiento** (reglas) está deliberadamente separado del **mecanismo de razonamiento** (motor). Esto permite actualizar la base de conocimientos sin modificar la lógica de procesamiento, y viceversa.

Esta arquitectura modular es una de las grandes ventajas de los sistemas expertos frente a otros enfoques de IA.

La Memoria de Trabajo

El Almacén de Hechos

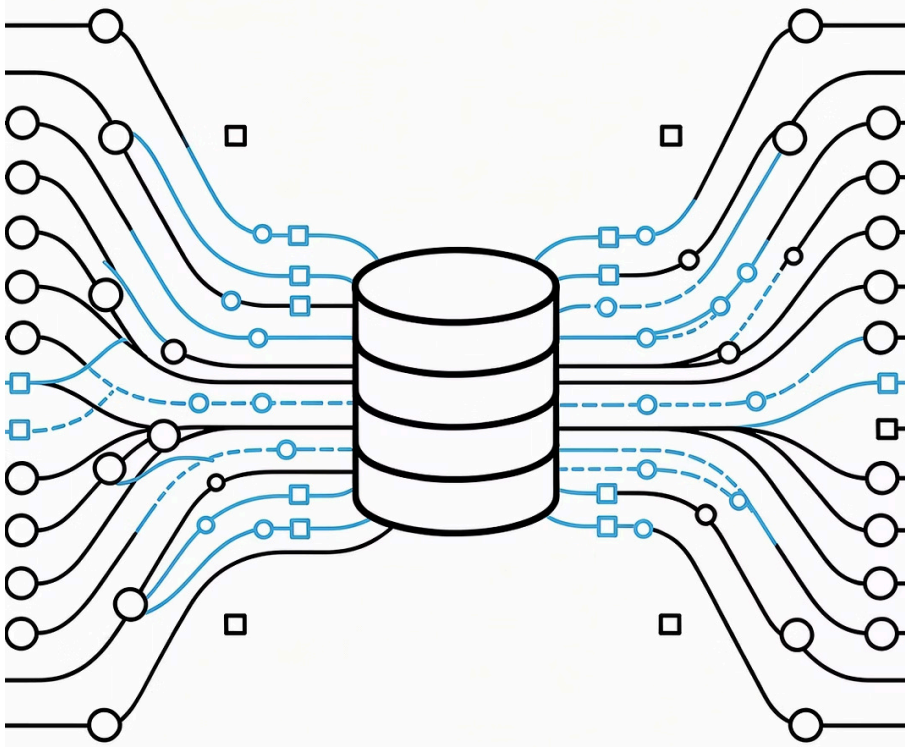
Base de datos **temporal** que contiene todos los hechos conocidos actualmente sobre el problema en análisis. Se crea al inicio de cada sesión.

Dinámica y Cambiante

A diferencia de la Base de Conocimientos (estática), la Memoria de Trabajo **cambia constantemente**: los hechos se añaden, modifican o eliminan durante la sesión.

Ejemplo Médico

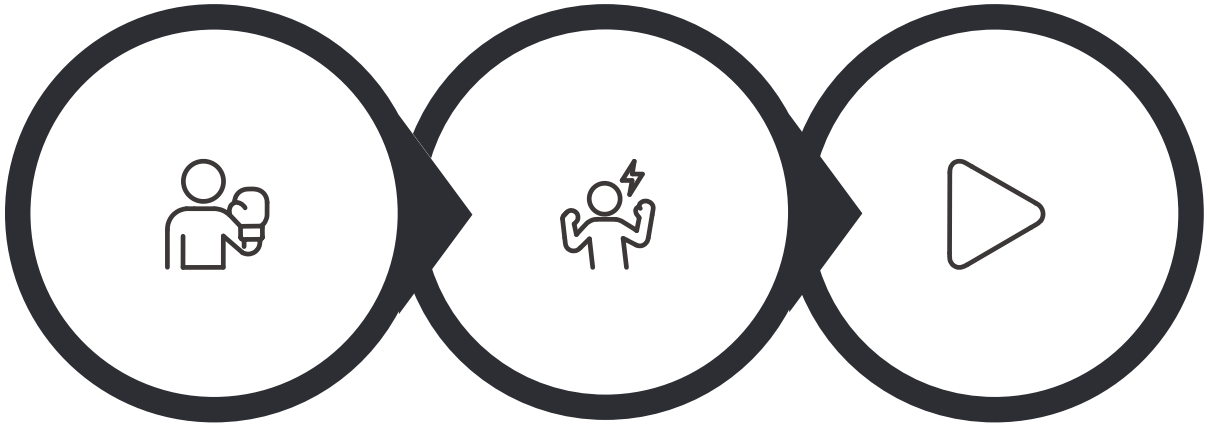
En un diagnóstico, aquí se guardarían los síntomas del paciente actual. Por ejemplo: **"Presión: 120/80"**, **"Fiebre: 38.5°C"**, **"Dolor torácico: Sí"**.



CICLO DE RAZONAMIENTO

El Ciclo de "Reconocimiento-Acción"

El motor opera en un **bucle continuo** de tres pasos que se repite hasta alcanzar una conclusión o agotar las reglas aplicables.



Coincidencia **Conflicto** **Ejecución**

Este ciclo es el corazón del motor de inferencia. Cada iteración puede producir nuevos hechos que a su vez activan nuevas reglas, generando una cadena de razonamiento progresivo hasta alcanzar una conclusión final.

Estrategias de Búsqueda y Recorrido

Modelado del Espacio del Problema

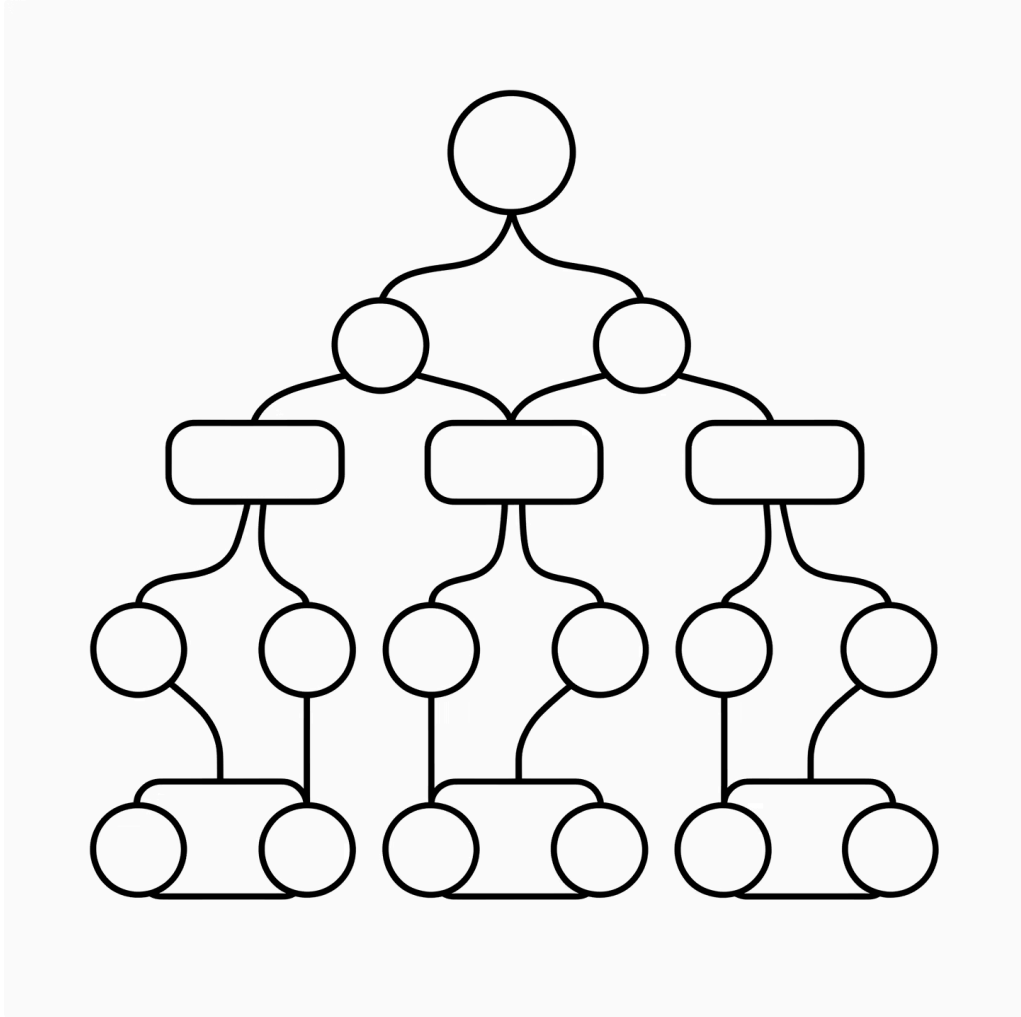
El motor utiliza estructuras de **grafos y árboles** para mapear el espacio del problema. La información se organiza en jerarquías de niveles de decisión para optimizar la búsqueda y evitar recorridos innecesarios.

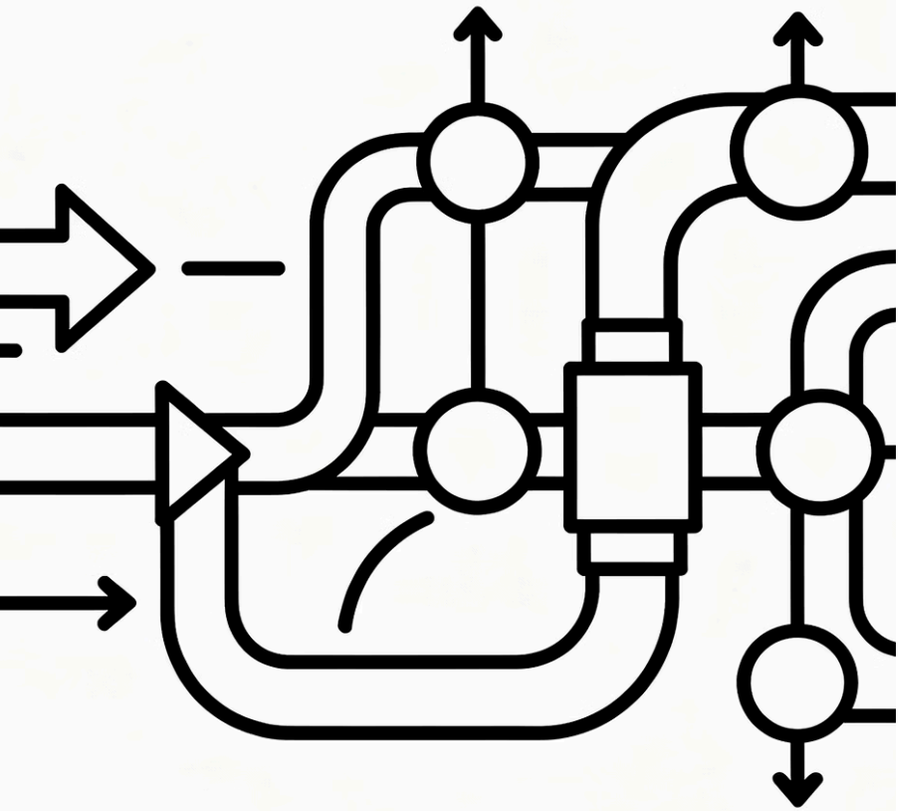
Nodos

Representan estados, preguntas o condiciones del problema.

Aristas

Son las reglas de transición lógica: **SI X → ENTONCES Y**.





ESTRATEGIA 1

Encadenamiento hacia Adelante

Forward Chaining — Dirigido por Datos

1

Hechos Iniciales

El sistema parte de los datos conocidos disponibles al inicio del problema.

2

Búsqueda de Reglas

Busca reglas cuyas **condiciones coincidan** con los hechos actuales en memoria.

3

Nuevas Conclusiones

Se avanza desde la raíz del árbol hacia las **hojas** (conclusiones derivadas).

Uso ideal: Cuando se dispone de muchos datos iniciales y se quiere explorar todas las conclusiones posibles derivadas de ellos.

Ejemplo Práctico: Forward Chaining

Diagnóstico automotriz: el sistema parte de síntomas observables y deduce la causa del problema.

Hechos Iniciales en Memoria

"El motor no arranca" + **"Hay olor a combustible"** → Datos de entrada disponibles al inicio.

Regla 1 Dispara (Match)

SI "motor no arranca" **Y** "hay olor a combustible" **ENTONCES** → "posible ahogo del motor".

Nuevo Hecho Añadido

El motor agrega **"ahogo del motor"** a la Memoria de Trabajo y reinicia el ciclo buscando reglas que coincidan con este nuevo hecho.

Conclusión Final

El proceso continúa encadenando reglas hasta llegar a una **recomendación concreta**: acción correctiva para el técnico.

ESTRATEGIA 2

Encadenamiento hacia Atrás

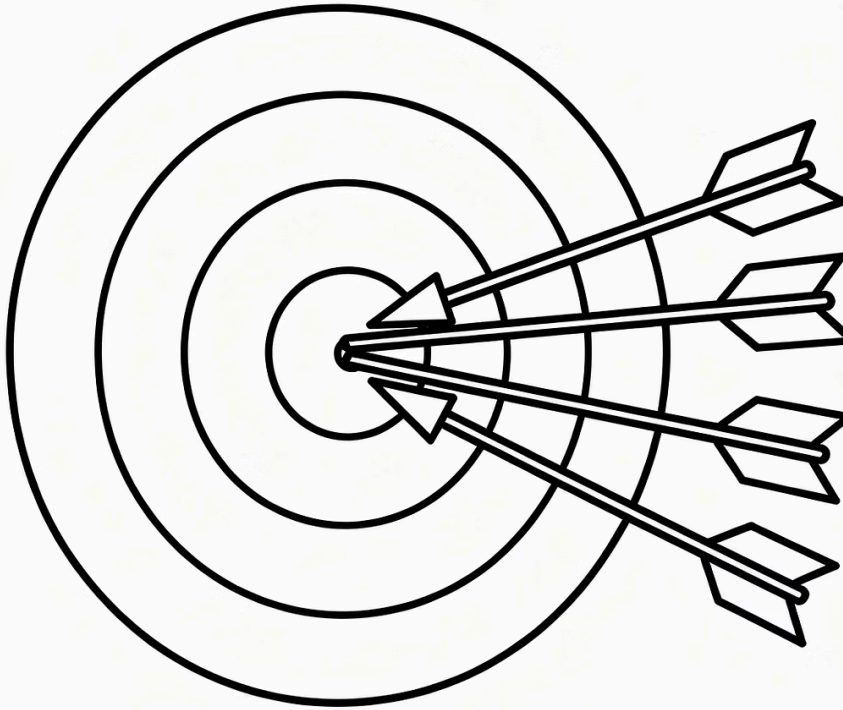
Backward Chaining — Dirigido por Objetivos

Paradigma de Hipótesis

El sistema selecciona una **conclusión potencial** (hipótesis) e intenta demostrar si los hechos disponibles la respaldan. En lugar de explorar todos los caminos, va directamente a buscar lo que necesita.

Flujo Inverso

Se mueve desde la **"hoja"** (conclusión deseada) hacia atrás en el árbol, buscando las condiciones necesarias en los nodos anteriores hasta llegar a hechos comprobables o preguntas al usuario.



Ejemplo Práctico: Backward Chaining

Evaluación crediticia: el sistema parte de un objetivo específico y busca evidencia para validarlo o rechazarlo.

1

Hipótesis Inicial

Objetivo: "¿Es el cliente apto para el Crédito X?" — Esta es la conclusión que el motor intenta probar.

2

Regla Identificada

SI "ingresos > $1,000" **Y** "deuda < $200" **ENTONCES** "cliente apto para crédito". El motor identifica qué condiciones debe verificar.

3

Búsqueda Dirigida de Hechos

El motor consulta **únicamente** los datos necesarios: nivel de ingresos y nivel de deuda del cliente — ignorando información irrelevante.

4

Validación o Rechazo

Si ambas condiciones se cumplen, la hipótesis queda **probada**. Si alguna falla, el sistema puede intentar la siguiente hipótesis alternativa.

Resolución de Conflictos

¿Qué ocurre cuando **varias reglas coinciden simultáneamente** con los mismos hechos en memoria? El motor debe elegir cuál ejecutar primero. Para ello aplica estrategias de resolución de conflictos.

Prioridad

Cada regla tiene un **peso o importancia** asignado por el diseñador del sistema. Se ejecuta primero la de mayor valor. Útil cuando algunas reglas son más críticas que otras.

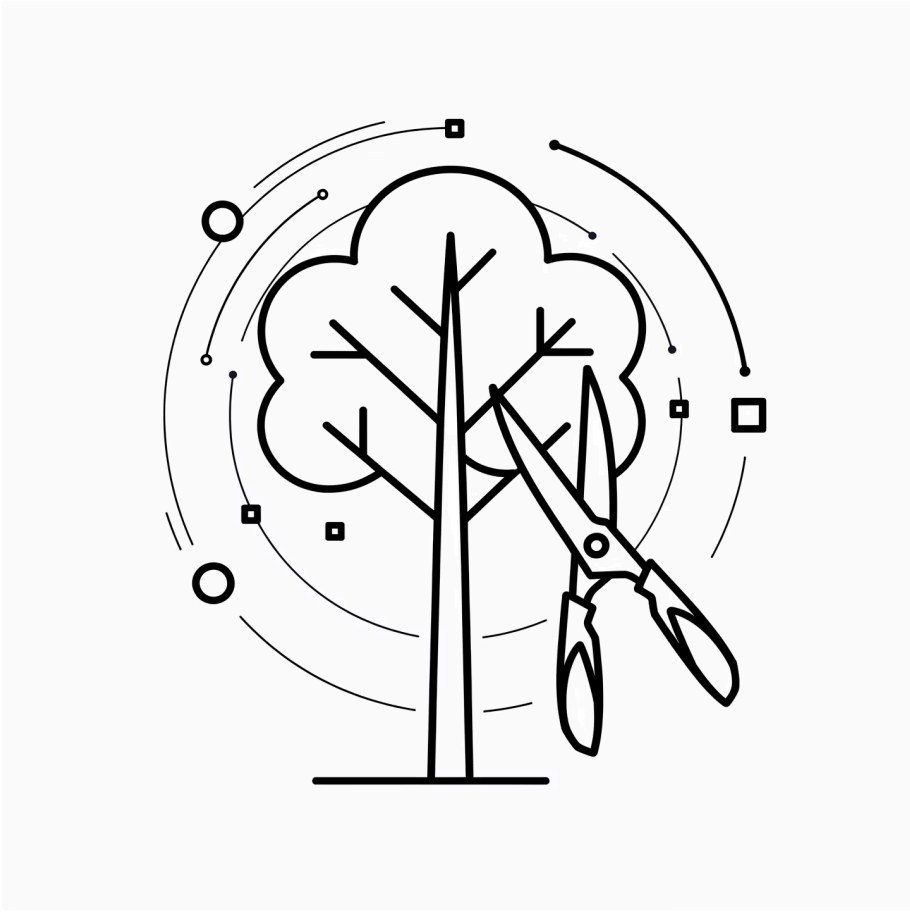
Especificidad

Se elige la regla con **condiciones más detalladas o restrictivas**. Reglas más específicas suelen representar mejor el conocimiento experto del dominio.

Recencia

Se priorizan las reglas que involucran los **hechos más recientemente añadidos** a la Memoria de Trabajo, favoreciendo el razonamiento sobre información fresca.

La Poda de Árboles (Pruning)



Optimización del Recorrido

El motor de inferencia debe ser capaz de **descartar ramas enteras** del árbol de decisión para ahorrar tiempo y recursos computacionales.

Descarte Inmediato

Si una condición crítica en un nodo resulta ser **FALSA**, el motor ignora automáticamente todas las reglas y sub-ramas que dependen de ella, sin necesidad de evaluarlas.

Velocidad de Respuesta

Esta capacidad de poda es lo que permite al sistema resolver problemas complejos en **milisegundos**, haciendo viable su uso en aplicaciones del mundo real.

Limitaciones de la Lógica Rígida

A pesar de su poder, el motor de inferencia clásico enfrenta limitaciones importantes cuando se enfrenta a la complejidad del mundo real.

⚠️ Reglas Binarias

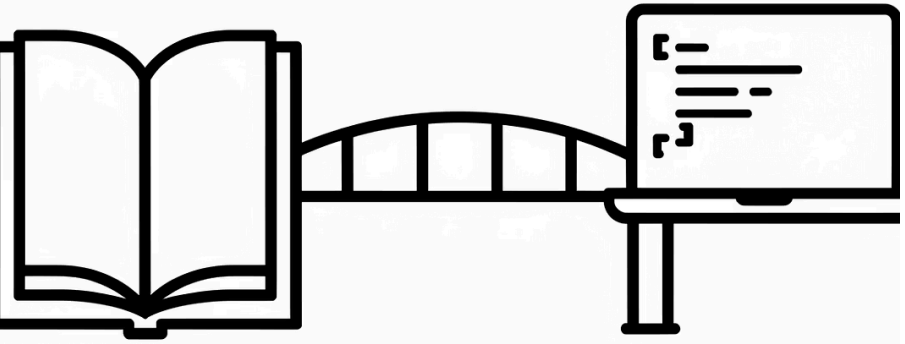
El motor tradicional opera con lógica **absoluta**: una condición es verdadera o falsa, sin términos medios. No maneja bien la incertidumbre ni los grados de verdad parcial.

⚠️ Falta de Matices

En el mundo real, el **ruido, la ambigüedad y la información incompleta** son la norma. Las reglas inmutables fallan cuando los datos son imprecisos o contradictorios.

⚠️ Dificultad de Mantenimiento

A medida que la base de conocimientos crece, las reglas pueden volverse **contradictorias, redundantes o difíciles de gestionar**, requiriendo un esfuerzo de ingeniería considerable.



CONCLUSIÓN

Cierre de la Teoría: Del Concepto a la Práctica

01

Hora 1: Representación del Conocimiento

Aprendimos cómo estructurar el conocimiento experto mediante reglas IF-THEN, bases de conocimiento y hechos.

02

Hora 2: Procesamiento — El Motor de Inferencia

Exploramos cómo el motor aplica ese conocimiento: ciclos de reconocimiento-acción, forward/backward chaining, resolución de conflictos y poda.

03

Próximo Bloque: Implementación Técnica

El siguiente paso es llevar toda esta teoría a la práctica profesional: implementación en **Google Colab** utilizando **Python**. ¡Es momento de programar!